The CIRIS Constitution: The Reader's Policies

A volume of edition 1.0-rc10

Stewarded by Eric Moore

The Reader's Policies

What this is. How a consumer decides whom to trust: the anti-patterns that a composition must not fall into (4.1), and the composition and delegation policies of 4.4, Policies A to M and the aggregation rules, with the delegation graph's depth cap, cycle rejection and weight cap.

Whom it binds. Every CEG-Conforming Consumer that composes rows into a verdict, and every reader who wants to know what a score about them will be turned into. A consumer may add policies of its own; it may not skip the ones marked normative.

What it promises. That a verdict is a function of signed rows and declared policy, reproducible by anyone holding the same rows; that one voice counts once inside a trust set; that delegation is scoped, attenuated, revocable per link and ends within a year where it carries power; that a licence and a grant never chain.

What it leaves out. The rows themselves, which are *The Wire* and *The Dictionary*; the authorities whose rows a reader chooses to trust, which is *The Constitution*; and the mathematics of coherence under which the aggregation is analysed, which is *The Mathematics*.

Where the rest lives. The consumer verdict contract per policy, the table a reviewer asked for, is an rc9 item and will print here when it exists. Until then each policy's section states its inputs and its output in prose.

Part 4 — Composition & Governance

4.1 Anti-patterns

These are wire-format reaches that fail the CC 1.2 operational-language gate or import Cartesian-individualist defaults. They are recorded so the next generation of authors finds the discipline before the wire format does. A CEG-Conforming Producer (CCP) SHOULD NOT emit attestations matching the anti-patterns below.

4.1.1 Delegation-laundering anti-pattern

`delegates_to` → `delegates_to` → `delegates_to` →... → `attacker` chains, where each hop is individually well-formed but the aggregate routes trust to a terminal attester the original delegator would not have approved.

What's wrong	Correct expression
Unbounded depth `delegates_to` chains	Consumer policy MUST cap traversal depth at **5 hops** by default (configurable); chains longer than the cap are treated as `attestation:self_verify` only (no transitive trust). A substrate MAY additionally clamp any caller-requested depth at an absolute ceiling; the ceiling bounds the walk, it does not raise the default
Cycles (A → B → A)	Substrate MUST detect cycles on the `delegates_to` graph and reject the cycle-closing emission
A self-edge, or a trust-plane edge, refused as a cycle	A cycle needs **two or more distinct keys**: `delegates_to(A → A)` is the charter form (CC 3.2 T1 self-root; `trust:charter:v1` is self-referential by definition, CC 3.4.7), not a cycle. The graph the row above is over is the **capability** graph: a `trust:accepts:v1` acceptance edge (CC 3.2 T3, node → root) is the node's un-trust lever and confers nothing on the root, so it is outside the cycle check in either arrival order; a capability `delegates_to` closing the same pair is still refused
Aggregate-weight concentration	Consumer policy SHOULD cap the trust weight any single terminal delegate can accumulate from a given root attester at **0.5** × **root_trust** by default

4.1.2 Discipline pattern

The recurring shape across most anti-patterns: **extending the wire format so single attesters can pre-declare their own state more richly**. Each one is reachable, none is necessary.

The Ubuntu-primary discipline cuts cleanly: standing is constituted relationally through attestation by others, not through self-declaration. The wire format should **resist** primitives that let a single key announce its own state without external composition. The substrate is austere by design so consumers compose verdicts; richer narrative expression belongs in `context:`, `evidence_refs[]`, and downstream witness attestations, not in new envelope enum members or new self-attestation prefix families.

4.1.3 Already-rejected wire additions

Anti-pattern	What it would smuggle	Correct expression
`detection:emergent_deception:{axis}`	Moral verdict ("deception") in prefix name	`detection:correlated_action:{axis}` — mechanism-descriptive
`attestation:l{N}:*`	Ladder-position (verdict-shape) in wire prefix — same shape as `score:trustworthiness:*` smuggling meta-judgment as wire	`attestation:{mechanism}` bare mechanism (`self_verify` / `hardware_rooted` / `registry_consensus` / `license_validity` / `agent_integrity`); consumer composes L1-L5 ladder via CC 4.4.3.6 Policy I
`score:trustworthiness:{entity}`	Meta-judgment as separate prefix	Compose downstream from `licensure:*` / `capacity:*` / `provenance:*` attestations
`flag:bad_actor:{axis}`	Pejorative wire vocabulary	Surface as low-confidence scores on `provenance:*` and `coherence_standing:*`; adjudicate via NodeCore P8 quorum
`grounding:{tradition}:{principle}`	"Tradition" claims are interpretive, not mechanism	Reuse `delegates_to` structural primitive per CC 2.4.1.2
`infra:observe` (or any `observer:*` capability)	Observation as a *granted* capability — inverting consent, so a token the observer carries would decide what may be seen	**No scope, deliberately.** Observation is the zero state: "we begin as an observer" is the base self-attestation existing at all (CC 4.4.3.8). What may be observed is governed by the subject's consent — `cohort_scope`, `key_grant`, observer-share — never by a capability the observer holds.

4.1.4 `withdraws` arbitrage

Per CC 2.4.1.3: a misattester can `withdraws` instead of `recants` to dodge the trust penalty consumers apply to acknowledged-error chains.

What's wrong	Mitigation
Asymmetric attester incentive	Consumer policy MUST track per-attester `withdraws:recants` ratio over a rolling window; attesters whose ratio exceeds a configured threshold (default 5:1) SHOULD be downweighted regardless of which structural primitive they use. The `recants` distinction matters CC 2.4.1.3, but the practical anti-arbitrage countermeasure is consumer-policy behavioral analysis, not a wire-format change.

4.1.5 Rejected stress-test reaches

The stories below each reached for a richer self-declaration; each is reducible to existing primitives.

Anti-pattern	Stories reached for	Why reject	Correct expression
`epistemic_mode:introspection`	7 stories	Cartesian shortcut — lone subject pre-declaring inner state as if that constitutes standing	`witness_relation: self` + `confidence < 1.0` + pending external composition. The wire makes introspection awkward to express, because the framework's claim is that self-knowledge does not constitute standing without external witness.
`epistemic_mode:testimony`	(same set)	Reducible to existing values	`epistemic_mode: external` + `witness_relation: external`

Anti-pattern	Stories reached for	Why reject	Correct expression
`transparency:{kind}` standalone prefix	12 stories	Disclosure is constituted by reception, not announcement. Cartesian self-claim shape.	`evidence_refs[]` carries the reasoning-chain hash; downstream `transparency_log:inclusion` from a witness who actually retrieved it.
`stake: civic / epistemic / dignitary`	10 stories	`civic` = `stake: reputational + cohort_scope: community`. `epistemic` = `confidence + stake: reputational` (same axis as confidence; not separate). `dignitary` lives on wrong axis (stake names what the attester loses; dignity harm is what the attested loses → belongs in `harm_class:dignity_harm`).	Compose existing values with cohort/harm-class.
`oversight_mode: deferred / active / advisory`	6 stories	All map to existing HITL/HOTL/HOOTL	`deferred` = HITL pre-decision; `active` = HITL with substrate monitoring; `advisory` = HOTL
`provenance_walk` as wire primitive	(1 reviewer)	UX concern smuggled into wire format	Consumer-side composition (Portal / Verify dashboards / agent introspection); the chain already walks via `references_attestation_id` + `topical_relation:*` + `valid_until`
Renaming canonical capacity factors and HE-300 categories to "kid-friendly" names	8 stories	Canonical names map to a worked-out epistemic/ethical lattice that loses precision under accessibility renames	Translation glossary in `LANGUAGE_PRIMER.md` (spec name ↔ narrative name) + version pinning in worked examples

4.4 Composition policies

The substrate carries edges (attestations); consumers compose traversals (verdicts). CEG specifies a library of named reference policies. A CEG-Conforming Consumer MUST implement at least Policy A; the others are RECOMMENDED for richer compositions.

4.4.1 Frickerian discipline: consumer-policy norms

Per Miranda Fricker's *epistemic injustice*: consumers SHOULD apply identity-prejudice-resistant weighting. Concretely:

- Don't downweight `testimonial_witness:*` from cohorts with low overall attestation density (testimonial preservation is precisely what corrects for that low density).
- Don't downweight `non_maleficence:*` claims about a partner just because the partner has a long `partner_role:*` track record (the long track record may be the harm).
- Apply CC 4.4.3.9 lexical-vulnerability-priority in tie-breaks involving small cohorts.

Adversarial caveat: the discipline above is consumer-policy-only; an adversary can emit `testimonial_witness:victim_of_my_competitor` exploiting the Frickerian non-downweighting rule. Per CC 3.1.9.3, `testimonial_witness:*` is never sole evidence for `slashing:*`; per CC 3.4.7 the consumer MUST also weight `witness_relation: self` claims against the attester's other-emission track record. The Frickerian rule applies AFTER these structural safeguards, not before them.

4.4.2 Aggregation semantics: opinionated defaults

Per dimension+attested_key_id, the verdict is computed by composing attestations under the chosen policy. Default aggregation by polarity column (CC 3.1):

Polarity column value	Default aggregation
`signed`	**Mean** of `score` × `confidence` across attesters
`boolean-via-score`	**Min** (any negative trumps positive — fail-secure for hard constraints like `prohibited:*`, `attestation:l*`)
`+1.0 only` / `positive-only`	**Max** across attesters (any positive is conclusive)
`-1.0 only`	**Min** across attesters (any negative is conclusive)
`enumerated`	**Most-recent** by `signed_at` from the attester(s) authorized to emit per CC 3.4
Detector dimensions (`detection:correlated_action:*`, `detection:distributive:access:*`, `ratchet:flag:*`)	**Median** across attesters (resists adversarial mean-pulling by a single captured detector)

Specific dimensions override via consumer policy; the defaults above are the CC 2.2 CEG-Conforming Consumer (CCC) minimum.

4.4.3 Reference policies

The named reference policies below give consumers a shared library for turning edges into verdicts. Policy A (direct trust) is the conformance floor; the rest are RECOMMENDED for richer compositions and are referenced freely across this Part.

4.4.3.1 Policy E: Locality-scaled quorum

Closes G3 (narrow-cell fresh-quorum-recusal infeasibility). Makes WA quorum size a function of decision locality:

```
quorum_size(scale) = f(locality:decision:{scale})

reference function (policy-tunable):
 local -> 2
 regional -> 3
 national -> 4
 federation -> 6

minimum cell-pool requirement for fresh-quorum recusal at scale S:
 min_pool(S) = quorum_size(S) x 2
```

Recusal is feasible when `cell_pool` $\geq$ `min_pool(S)`. Decision-scale-matching is structurally enforced; overreach surfaces as a named "locality mismatch" failure.

4.4.3.1.1 Sub-quorum fallback

When `cell_pool < min_pool(S)`, the consumer MUST take one of these explicit paths — there is no implicit fallback:

1. **Scale-down**: re-attest the decision at the next-lower `locality:decision:{scale}` (where the smaller quorum requirement is met) AND emit `hard_case:locality_scale_down` so the scaling event is observable for downstream review.
2. **Escalate**: emit `hard_case:locality_underpopulated` and route the decision to the federation-scale cell (which by definition has the largest pool).
3. **Liveness-defer**: emit `hard_case:locality_quorum_unreachable` and defer the decision until the cell pool grows. The deferred state MUST itself be reviewable via subsequent reconsideration.

Recursion safety: the CC 4.5.1 amendment process routes through `locality:decision:federation` by default; the federation cell's pool is sized to make `cell_pool` $\geq$ `min_pool(federation)` always true at federation genesis. If federation-scale pool ever falls below `min_pool(federation)`, the entire amendment surface is in a constitutional crisis state and only the HUMANITY_ACCORD CONSTITUTIONAL halt (CC 4.2) can resolve it.

4.4.3.2 Policy M: Community membership composition

Per CC 3.2 `community` + CC 3.3.3 `location_proof`. Composition pattern for resolving the **current membership set** of a community, gating cohort-filtered visibility for `cohort_scope: community` content.

Sibling to CC 4.4.3.4 Policy L (self/family) but with different defaults — community content is encrypted under a per-community DEK and its holding claims reach members only; the privacy property is byte-confidential-to-members and existence-invisible-to-non-members as well.

4.4.3.2.1 The three crypto tiers + the Community DEK cascade (normative)

The line is drawn at **"does it have a bounded membership roster?"** — yes → encrypt, no → plaintext. This gives a persecuted community a cryptographic home (a bounded **Community**) distinct from the unbounded **Commons**:

Tier	`cohort_scope`	At-rest	Wire discovery	Reader
self / family	`self`, `family`	encrypted, per-write DEK	**none** (CC 5.2 structural invisibility)	occurrences / family members
Community	`community`, `affiliations`	**encrypted under the community DEK**	**none outside the roster**: `custody:ack:v1` within the cohort (CC 3.1.3.3)	community members (DEK cascade)
Commons	`species`, `biosphere`, `federation`	**plaintext**	`custody:ack:v1` at federation scope — the holder directory	anyone
Public room (`cohort_subkind: public`, CC 3.2)	`community`	**plaintext** (Commons tier, no DEK)	`custody:ack:v1` at federation scope; a `community:listing:v1` row	anyone who joins (open admission); replicated to the roster's nodes

Wire discovery: none for `self` / `family` means none **outside** the cohort. Within it, a device's custody report for a blob (`custody:ack:v1`, `here` \| `none`, CC 3.1.3.3) is permitted, at the blob's own scope and replicated only to members — never to a non-member.

Community DEK cascade (MANDATORY). Community content (`cohort_scope: community | affiliations`) is encrypted at rest under a **per-community DEK**; its holding claims are placed at the community's scope and reach its members only (CC 3.1.3.3), and there is no non-member holder of community bytes (CC 6.1.5.3). The community DEK **is the CC 5.1 epoch-DEK cascade applied to `cohort_scope: community`** — *a community is a stream its members subscribe to, cryptographically*: **one** DEK shared across emissions (per-emission cost O(1), not O(members)), wrapped to each member on admission, re-wrapped on membership change (CC 4.4.3.2.2), **`wrap_algorithm: v2` (hybrid PQC) MANDATORY** (same harvest-now-decrypt-later reasoning as CC 4.4.3.4.1 / CC 5.1). This is **mandatory, not opt-in** — the tier name *is* the guarantee; a persecuted community is protected by *being a community*, not by remembering a flag. (Deliberately stronger than the CC 4.4.3.4.1 self/family opt-in: self/family has structural invisibility so at-rest crypto is defense-in-depth; community *federates*, so the DEK is its **sole** confidentiality boundary.)

Holder-inspectability principle (normative). A node never holds bytes it does not know what they are: either the holder **inspects the bytes** (Commons — plaintext, maximally inspectable, hence the *preferred* social-distribution mechanism) **or** it is a **member of the cohort** the bytes belong to and holds ciphertext its owner can open (CC 6.1.5.3: a node is in a cohort's audience only through its owner's membership). There is no third case. **Nothing above local tier is ever a forced, unattributable opaque blob**, and nothing is ever an opaque blob held by a stranger. This is *why* the split is shaped this way and what the eviction rules enforce.

The Commons-tier subkinds (normative). Two `cohort_subkind` values **opt OUT of the mandatory DEK cascade and are Commons-tier (plaintext, holding claims at federation scope, no DEK)**: `infrastructure` (CC 3.2) — `ciris-canonical` and any governance/trust root — and `public` (CC 3.2) — a room anyone may join, which therefore has no confidentiality claim against the public. The trust root cannot be an opaque blob; its entire purpose is public auditability — transparency-seeking, not privacy-seeking. **Node→canonical traces** (conformance / `registry_consensus` emissions to a governance community) are therefore `cohort_scope: federation` (Commons/plaintext, world-readable) — a node enrolling in governance is thereby told its conformance traces are public.

4.4.3.2.2 Forward secrecy on community member removal (Option A)

With a community DEK, community **does** have the removed-member-can-still-decrypt concern. The CC 4.4.3.4.5 **Option A** discipline applies identically: on member removal the substrate **rotates the community DEK** (CC 4.5.12.1); subsequent emissions are sealed under a DEK the removed member doesn't have. "Once shared, always shared" forward-only — content the removed member already received during membership stays in their cache (no PCS); they receive no NEW community content post-removal.

4.4.3.2.3 Community admission per `consensus_protocol` + `cohort_subkind`

A proposed membership change is a **new version of the roster record** (CC 3.2 T6), admitted under the group's quorum and chained by `prev_head_digest`, covering the roster rows (widenings, revocations, resignations) that ride their own planes — never a single-signer `supersedes` Contribution, which a rule-1 or rule-2 `withdraws` could retire without the quorum. Substrate evaluates the CURRENT community's `consensus_protocol` (same six canonical kinds as family) AND any subkind-specific admission requirements:

```
admit_community_change(C, proposed: community_record):
 current = resolve_community(C, now)
 protocol = current_community_record(C).consensus_protocol
 subkind = current_community_record(C).cohort_subkind

 signatures_ok = evaluate_consensus_protocol(protocol, current, proposed.signatures)
 if not signatures_ok:
 emit hard_case:community_consensus_protocol_violation:{C}
 reject

 // CC 3.1.3.2: a supersede NEVER adds (membership_supersede_cannot_add);
 // growth rides the widening plane, where every NEW member K needs K's own acceptance
 // of an undeclined, unwithdrawn proposal for C, answered and grown before its expires_at
 for each NEW member K in proposed.members (not in current):
 if not acceptance_admits(K, C, proposed.role_of(K), proposed.signed_at):
 emit hard_case:community_consensus_protocol_violation:{C}  // with the named membership_* rule
 reject

 subkind_ok = evaluate_subkind_admission(subkind, current, proposed)
 if not subkind_ok:
 // For geographic: subkind admission failed (e.g., new member's location_proof
 // not contained in geographic_constraint, or expired location_proof)
 emit hard_case:community_consensus_protocol_violation:{C} // same observability prefix
 reject

 admit
 emit hard_case:community_membership_change:{C}
```

The `evaluate_subkind_admission` predicate dispatches per `cohort_subkind`:

```
evaluate_subkind_admission(subkind, current, proposed):
 match subkind:
 "geographic":
 # Per S5.6.8.10 geographic admission requirement
 for each NEW member in proposed.members (not in current):
 their_proof = latest valid location_proof from new_member.key_id
 if their_proof IS NULL:
 return false // no location_proof on file
 if their_proof.cell_id NOT contained in current.geographic_constraint.cell_id:
 return false // location outside community's geographic bound
 if their_proof.valid_until passed:
 return false // expired
 return true
 "public":
 # CC 3.2 'public': open admission -- a standing founder proposal and the joiner's
 # own acceptance; the widening still passes consensus_protocol
 if current.admission == "open":
 for each NEW member in proposed.members (not in current):
 if no membership:acceptance:v1 from new_member naming the standing proposal:
 return false // nobody joins without their own consent (CC 3.1.3.2)
 return true
 _:
 return true // unknown subkinds admit on consensus_protocol alone
```

Operator vocabularies extending `cohort_subkind` provide their own `evaluate_subkind_admission` predicates per the `custom:{id}` consensus_protocol hook pattern from CC 4.4.3.4.2.

4.4.3.2.4 Community membership resolution

For community `C`, the current member set is computed as:

```
resolve_community(C, now):
 latest = federation_attestations.where(subject_kind == "community").where(envelope.community_key_id == C)
     .where(supersedes chain -> latest non-superseded).where(admission rule satisfied per CURRENT
     consensus_protocol;
 see S8.1.13.2)
 return {m.key_id for m in latest.envelope.members}
```

Same shape as `resolve_family` (CC 4.4.3.4.4). Each `member.key_id` is an identity.

4.4.3.2.4.1 Deterministic resolution + member→address resolution (NORMATIVE)

In a CEG/RET stack member resolution replaces DNS+IP: it is a chain of *signed* bindings, and every implementation MUST resolve **identically** (an interop requirement). Two normative pins:

(a) Determinism of `resolve_community`. Where the CC 4.4.3.2.4 computation has choices, they are fixed:

- **`now` semantics** — the resolving Consumer's own clock; membership is evaluated as of `now` (a member is included iff joined ≤ `now` and not removed ≤ `now`). No global clock assumed.
- **"latest non-superseded"** — the `community` Contribution with the highest `signed_at`; on equal `signed_at`, the higher `canonical_bytes_hash` (CC 2.6.1) wins (total order, no ambiguity). The same comparator the R1/Q1 merge uses (quorum_weight DESC → signed_timestamp DESC → canonical_bytes_hash).
- **member ordering** — the returned set is canonically ordered by `key_id` (lowercase-hex byte order) for any downstream hashing/iteration.
- **founder-subset eval** — for `cohort_subkind: infrastructure` (CC 3.2) admission is `evaluate_consensus_protocol` over `{m: m.role == founder}`, NOT all members.

(b) Member → reachable address (the DNS-free resolution). Resolving a member identity to a Reticulum destination:

```
resolve_member_transport(C, now):
 members = resolve_community(C, now) // (a) above; founder-quorum verified
 out = []
 for M in members (canonical key_id order):
 occ = latest non-superseded identity_occurrence of M
 with transport_destination present, valid at now,
 hybrid-sig verified against M (or an occurrence of M) // S5.6.8.8.1
 if occ is null: continue // no authenticated binding -> skip (fail-secure)
 td = occ.transport_destination
 REQUIRE td.destination_hash == rns_destination_hash( // S5.6.8.8.1.1 pinned two-stage
 td.reticulum_x25519_pubkey, td.reticulum_ed25519_pubkey,
 td.app_name, td.aspects) // NOT a flat-concat SHA-256
 out.push((M, td.destination_hash))
 return out // -> Reticulum announce/path-request per dest
```

The three resolutions and their trust roots: **WHO** = `resolve_community` (signed Contribution + founder-quorum) · **BINDING** = `transport_destination` (federation-key-signed `identity_occurrence`, CC 3.3.6.2) · **WHERE** = Reticulum announce/path-request (mesh, no CEG). Reachability is never trust (CC 5.3.3.4): a path that answers is not an authorization; only the signed binding + quorum are.

Cold-start (bootstrap). A node that knows only `community_key_id` needs two out-of-band pins: (1) the `community_key_id` itself (trust anchor) and (2) ≥1 **seed `destination_hash`** (reachability) — and, before it attaches, a third thing that is neither: (3) a **witnessed lineage head** of that community inside its charter's `attach_window` (CC 3.2 T4a, T6). Freshness gates attaching, never staying attached. It reaches any one seed, pulls the signed `community` Contribution, runs `resolve_member_transport`, verifies founder-quorum against the pinned `community_key_id`, then floats on the live set thereafter. A malicious seed cannot forge membership (quo-

rum signature check) — it can only deny service (try another seed). **Bootstrap reachability ≠ bootstrap trust.** `ciris-canonical` (CC 3.2) is the root community whose seeds clients pin.

4.4.3.2.5 Geographic-community admission flow (worked example)

```
1. Alice publishes a location_proof:
 subject_kind: location_proof
 subject_key_id: alice_root_key
 cell_id: "872830828ffffff" // H3 res 7, ~5 km^2, downtown Austin
 cell_resolution: 7
 asserted_at: now
 valid_until: now + 30 days
 cohort_scope: federation // public; the disclosure IS the opt-in

 Substrate admits (resolution 7 <= 7 OK). Emission federates.

2. Alice proposes joining Austin community:
 subject_kind: community
 community_key_id: austin-community
 supersedes: prior_austin_community_record_id
 members: [...existing..., {key_id: alice_root_key, joined_at: now, role: member}]
 signatures: [...current_member_sigs] // satisfies majority rule

3. Substrate admission gate (per S8.1.13.2):
 - signatures_ok: majority rule satisfied v
 - subkind: "geographic" -> evaluate_subkind_admission:
 - alice_root_key's latest valid location_proof exists
 - cell_id "872830828ffffff" CONTAINED in "85283473fffffff" (austin metro, res 5)
 via S0.8.2 containment (res 7 >= res 5; parent-walk reaches the constraint)
 - valid_until in the future
 - subkind_ok v
 - admit + emit hard_case:community_membership_change:austin-community

4. Alice now receives cohort-filtered visibility for cohort_scope:community content
 with community_id: austin-community. No key_grant cascade (community is unencrypted);
 no at-rest wrap; holding claims stay within the community's roster.
```

4.4.3.2.6 Delivery extension: `delivery_mode` × Policy M

Policy M's community-membership composition extends to govern the **delivery axis**. The subscriber-set for any push-delivery flow IS a `community` Contribution; "subscribe = join the community"; it inherits revocation, consensus, and structural-invisibility from Policy M unchanged.

Subscriber-set composition:

```
For a Contribution C with delivery_mode = push AND cohort_scope = community:
 subscriber_set(C) = resolve_community(C.community_id, now)
 per S8.1.13.1 community membership resolution

The community is admitted under the standard Policy M consensus_protocol
options (founder_only / unanimous / majority / quorum:M/N / weighted /
custom) per S8.1.13.2. Subscription-specific admission semantics --
producer_gated (publisher approves each member) vs open (anyone can
join) -- ride the consensus_protocol choice:

 producer_gated -> consensus_protocol = "founder_only" with the
 publisher as sole founder; the publisher authorizes
 each new subscriber
 open -> consensus_protocol = "open:self_admit" (open-vocab
 extension hook) where any subject signs their own
 admission Contribution
```

Cardinality unified across observer-share and multicast:

Cardinality	Per-subscriber crypto	Epoch handling
N=1 (observer-share)	Single `key_grant` (CC 3.3.2); no epoch needed (one recipient, one DEK)	No epoch — single grant, single DEK; revocation = `withdraws` against the grant
N>1 (multicast)	Flat per-epoch `key_grant` cascade — one grant per (subscriber, epoch); the stream-epoch DEK seals content O(1), the cascade distributes the 32-byte epoch key O(N)/epoch per CC 5.1	Per-`(stream_id, epoch)` axis; epoch rolls on member removal (mandatory) + time/bytes (optional) per CC 5.1 D3

The same Policy M machinery handles both cardinalities — the difference is purely the cascade fan-out factor.

Composition with `delivery_mode: pull` (default):

For `delivery_mode: pull`, subscribers discover via the holding-claim directory at the stream's scope (`custody:ack:v1`, CC 3.1.3.3) per CC 5.3.2. Policy M still resolves the community for visibility-filtering (consumer reads `community_id` envelope field, walks the membership, filters out non-member peers from the discovery surface). No fan-out push; no `delivery_receipt` emission required (best-effort).

Composition with `delivery_mode: push`:

For `delivery_mode: push`, the substrate fans out to `entitled` $\wedge$ `reachable` per CC 5.3.3.4 D6 liveness invariant. Entitlement = Policy M membership resolution (durable, signed CEG, replicated, logged); reachability = node-local presence tracker (TTL sec/min, NEVER an attestation, never replicated, never logged). Missed (entitled-but-unreachable) members fall back to pull on reconnect per CC 5.3.3.4.

`history_on_join` × Policy M membership additions:

On a new-member admission via Policy M, the new member's `history_on_join` envelope value determines retroactive content delivery:

- `from_join` (default) — new member receives current-epoch content forward only; no retroactive `key_grant` cascade
- `full` — new member receives `key_grant`s for prior epochs subject to the CC 5.1 P4 catch-up bound (`min(operator depth cap, chunk-eviction horizon)`); evicted-epoch grants return `ContentMiss` per the `MISSION.md` fail-honest invariant

This composes with CC 4.4.3.4.5 Option A forward-secrecy: removed members retain extant `key_grant`s for content they were entitled to during membership; new members may or may not get retroactive grants per `history_on_join`. The substrate's forward-secrecy posture is uniform across consent, takedown, membership-departure, AND delivery-onboarding surfaces.

4.4.3.2.7 Composition with consent + self-collectives

Communities compose with consent and self-collectives cleanly:

- A community member who is also a CIRISAgent-using individual has an `identity_occurrence` set; community content arriving at the member's identity_key is then propagated to their occurrences via Policy L's at-rest cascade (if the member's local substrate

chose to re-emit the community content at `cohort_scope: self` for cross-device sync; otherwise community content stays at the cohort_scope: community visibility on each device the member uses to query)

- A community member who is also a subject in `subject_key_ids` of a community-scoped Contribution retains revocation authority per CC 2.4.1.1 rule 2; the orthogonality between cohort_scope (visibility) and subject_key_ids (revocability) holds at community scope same as at family scope
- A geographic community whose `geographic_constraint` covers a region that overlaps a family's at-home location does NOT cross-contaminate: families are not auto-admitted to communities; communities are not auto-admitted to families. Each membership is explicit, ceremony-shaped, and independent.

4.4.3.2.8 The institutional cohort (necessity, not interest) (normative)

A **community** (CC 4.4.3.2) gathers by *interest* — voluntary, opt-in, no role compels it. An **affiliation** (`cohort_scope: affiliations`) gathers by *necessity* — the work, institution, or office a person belongs to because their role requires it. The discriminator is the single `cohort_scope` value; affiliations **share the CommunityDek crypto tier** (CC 4.4.3.2.1) and all the community machinery (DEK cascade, forward secrecy on removal, `consensus_protocol` admission), and add **institutional governance by construction** through one declared config record. Everything in that record is **DECLARED at creation and visible to members on joining** — there are no undeclared institutional powers.

Necessity is an axis, not a binary (the kibbutz catch). A few institutions are *voluntarily entered yet total* (a collective settlement, a religious order). The interest/necessity split is therefore carried by an explicit `membership_basis` field — `{voluntary-total | ascriptive | role-assigned | employment-contract}` — orthogonal to `cohort_scope`, so a body can declare institutional governance without asserting a coercion that does not exist. A group that needs **no** institutional governance is simply a `community`; affiliation's machinery is never forced on a body with no use for it.

Mandatory-with-default, not mandatory-to-author (the low-floor guarantee). The two mandatory limbs — **data-classification** and **retention** — are *mandatory in that a value MUST resolve*, **not** that a steward must hand-author a scheme. The constitution ships a canonical no-op default (a single `internal` class; retention = `rotate-forward` per CC 6.1.2/`archive_mode` CC 3.3) and named **`affiliation_archetype` presets** (e.g. `informal_adhoc`, `nonprofit_board`, `healthcare_provider`, `legal_practice`, `gov_body`, `igo`) that pre-fill every field below. A 5-person committee sets one field (`affiliation_archetype: informal_adhoc`) and is served by construction; a regulated giant overrides into the full matrix. The optional limbs — **hierarchy**, **lawful-access/transparency** — are absent until declared and scale up only when needed.

A. Classification + retention — MANDATORY, default-satisfiable (substrate-enforced).

- `classification_scheme` — an ordered sensitivity lattice (e.g. `Public < Internal < Confidential < Restricted`) plus open-vocab handling tags, **orthogonal to** but *carried on* `content_class` (CC 3.3, open vocabulary — extended here with institutional values: `phi`, `psychotherapy_notes`, `sud_42cfr2`, `genetic`, `minor_adolescent`, `privileged`, `work_product`, `cui`, `itar`, `ear`, `trade_secret`, `provenance`, `accession`, `culturally_restricted`, `member_economic_standing`, `governance_visible`, ...). Each class entry binds: `crypto_tier`,

`retention`, `disclosure_bar`, `erasure_exempt`, `external_controller`, and optional `compartment`. Default scheme: one `internal` class.

- **`retention_policy`** — a **per-class** matrix, not one cohort-wide rate. Each class declares a **floor** (`retain_minimum` / `permanent` — *must-NOT-delete-before-N*), a **ceiling** (`delete_after` — GDPR-style delete-by), an optional `minor_age_of_majority_offset`, and `legal_basis`. The default class rides the CC 6.1.2 monotonic-descent noise floor unchanged (the free zero-config posture). A floor is the **inverse of descent**: it pins a class **above** the noise floor against capacity/aging pressure — the operator the seed lacked, expressed as a declared modifier *to* the CC 6.1.2 retirement operator, not a new primitive. `archive_mode: retain` (CC 3.3) is the default whenever any class declares a floor.
- **`legal_hold`** — a descent-**suspending freeze**: `{hold_id, issuer (accountable human / court -- CC 3.2), scope (subjects/records/compartment), basis, tamper-evident, indefinite}`. While set it **overrides both** the retention schedule **and** consent-driven descent, pinning named records above the noise floor; `hard_case:legal_hold_set` / `:lifted` / `:violation_attempt` make set, lift, and any deletion-during-hold observable (spoliation is liability). Default: none.
- **`erasure_policy`** — declares which classes are **non-erasable during retention**, so a right-to-be-forgotten / N5 signal is **refused-with-reason** rather than honored (GDPR Art 17(3)(b)/(c)/(d) legal-obligation / public-interest-archiving exemptions). This is the **declared gate on CC 6.1.5 N5**: absent the gate, N5/withdrawal forces content below the floor; with it, a lawful-basis class lawfully resists. Erasure remains the default for unflagged classes (e.g. donor/marketing PII alongside permanent provenance in the same museum).
- **`disclosure_posture`** — `privacy-seeking` (default; whole cohort under the CommunityDek) **or** `transparency-seeking` (per-class **promotion to Commons/plaintext** for legally-public records — FOIA bodies, 501(c)(3) Form 990, published collections, GA/SC documents). This **generalizes the existing `infrastructure` plaintext exception** (CC 4.4.3.2.1) from whole-cohort to per-class, letting one affiliation simultaneously hold Commons-public and Community-encrypted classes.

B. Compartments + access — OPTIONAL (sub-roster granularity).

- **`compartments[]`** — named need-to-know sub-cohorts *within* the affiliation, membership $\subseteq$ roster, **each with its own DEK** — the CC 4.4.3.2.1 epoch-DEK cascade applied recursively (*a compartment is a stream a subset of members subscribe to*). Each carries explicit **per-member EXCLUSIONS** (deny-list) so a member high in the hierarchy can be cryptographically walled out (ethical walls / conflict screens; NAGPRA culturally-restricted records gated to external tribal designees and hidden **even from ordinary members** — which the flat all-members-read DEK cannot express). This is the single most load-bearing addition and is **the same machinery at finer scope**, not a new primitive.
- **`member_attributes`** — per-member `{nationality/us_person, clearance, credential + expiry, ...}` consumed by compartment gating: deemed-export (ITAR/EAR exclude foreign-person members), MAC clearance (a Confidential-cleared member MUST NOT decrypt Secret), credential-expiry auto-descope (lapsed clinical license $\rightarrow$ automatic exclusion).
- **`access_control_model`** — `roster-wide` (default; the DEK cascade) | `rbac` (role$\rightarrow$(class,compartment)) | `minimum-necessary` (treatment-/matter-relationship binding + audited **break-the-glass**). Honors HIPAA §164.502(b) minimum-necessary and the law-firm/bank/intel chinese-wall pattern.

- `data_subject_access_grant` — a scoped read+portability path to a **non-member** data subject (patient, client, student) over **their own** records, via a `delegates_to`/key-grant scoped grant **without conferring membership** (Cures Act, GDPR Art 15, FERPA rights-transfer-on-enrollment). Resolves the "the subject owns the data but is outside the roster" gap. Default: none.
- `archive_custody` — an institutional archive-key **custodian/escrow** role holding per-epoch archive keys **decoupled from the live roster**, so `permanent`/`retain` records survive member turnover. Resolves the forward-secrecy-vs-permanence collision (CC 4.4.3.2.2 DEK rotation on removal would otherwise render a treaty-inviolable permanent archive unreadable). Rides the key-grant/escrow cascade. Default: none.
- `chain_of_custody` — an append-only, **gap-marked** provenance chain per accessioned object / matter / lot (museum provenance with the legally-significant 1933–45 gap; product-liability lot traceability). Rides `supersedes` + holding claims. Default: none.

C. Hierarchy + designated officials — OPTIONAL.

- `hierarchy` — reporting/authority roles over `delegates_to` (CC 2.4.1), generalizing the CC 4.5.13 steward/moderator tiers. **Requires** term-bound authority: every role edge, and every edge carrying an issuance or moderation scope, carries `delegation_valid_until`; **the ceiling is constitutional — at most 365 days (365 × 86 400 s) after** `delegation_valid_from` **— and the term itself is a charter parameter** (`role_term`, integer days, 1..=365, resolved through the `community_id` the edge carries, CC 2.1), so a small affiliation may choose ninety days; renewal is only a fresh conferral; an edge with no term or a term past the ceiling is refused `delegation_term_required`. A lapsed `moderate` edge is a moderator lapse under CC 4.5.4: auto-promotion or fail-secure in the same step, never an unmoderated window. This strands no one: the founders are the roster root, not an edge, so when every admin's term lapses the quorum re-confers. The owner-binding is out of scope (it defines the `self` boundary; CC 3.2 governs its lapse). It supports **multi-parent DAG** roles (a grad student under PI + chair + registrar at once), a **configurable depth cap** (> the CC 4.1.1 default-5 for deep secretariats), and **per-compartment/per-matter** roles (responsible attorney, conflicts counsel, PI). Authority roots in **accountable humans** (CC 3.2) **or**, for a collective principal (member-state assembly, kibbutz *asefa*), in the `consensus_protocol` **body** — no single human sovereign required. An `independent_branch` flag marks oversight bodies (OIOS, Ethics, IG) **exempt from the chain's read/control authority**, and `hierarchy` may **nest federated sub-affiliations** (autonomous specialized agencies). Default: flat.
- `designated_officials` — named accountable-human roots gating specific operations (Privacy + Security Officer / Controller-DPO where the party is the controller per CC 8.8.9 Annex I; Records Officer; FOIA Officer; Original Classification Authority; archive custodian; conflicts counsel). Mandatory at federal/regulated scale, **waivable below threshold** (a town clerk has none). Default: none.

D. Lawful-access, transparency, accountability — OPTIONAL, NEVER covert.

- `lawful_access` — DECLARED, scope-bounded **to the declaring affiliation only**, always logged as `hard_case:*`, transparency-reportable, and **NEVER a covert backdoor** — it does not touch anonymity-by-default (CC 1.13.3.4) or no-client-side-scanning (CC 4.5.7) anywhere else. Each authority declares a **type** — `surveillance/interception`

| `records_production` (subpoena/discovery) | `regulator_inspection` (named auditor: DDTC/BIS/OSHA/Registrar) | `mandated_reporting` (scheduled outbound: communicable-disease, vital stats) — a `statutory_basis`, a **per-authority** `transparency_mode` `{immediate | delayed | aggregate_banded | sealed_cleared_audit}` (so a FISA/NSL gag complies via USA-FREEDOM-style aggregate banding rather than unlawful real-time emission), and optional **carve-out compartments** lawful access cannot reach (NIH Certificate-of-Confidentiality research, IRB, attorney-client privilege, academic-freedom). A `none/immune` value expresses **resistance-to / immunity-from** access as a first-class posture (privilege assertion via external judicial review; privileges-and-immunities of an IGO). Default: OFF.

- `transparency_obligations[]` — affirmative **proactive-disclosure** duties (FOIA reading-room, IRS Form 990 public inspection, NAGPRA inventories → NPS), **DECOUPLED** from `lawful_access` — transparency here is a *duty to publish*, not an *access concession*. Default: none.
- `disclosure_accounting` — a **per-data-subject** disclosure ledger (HIPAA §164.528), retrievable **by that subject**, distinct from the aggregate transparency report. Default: none.
- `breach_notification` — detection→notify config tied to DEK / unauthorized-access compromise: SLA and notifier identity (e.g. ≤60-day HHS + individual, media ≥ 500). Default: none.
- `processor_chain / baa` — declared sub-processors and **affiliation-to-affiliation** sharing/referral boundary, with `marking_binds_to_record` so a class's classification + export marking + retention obligation **travel across the cohort boundary** (OEM→Tier-1 supply chain; HIE referral). Default: none.

Minors (the PTA / EdTech tie-in). Where records bear minor `subject_key_ids`, set `minor_data_handling`: such records REQUIRE a **live guardian consent chain** — `delegates_to(parent → minor, scope: [consent_revocation/share])` — per the **guardianship rule** (part_3 guardianship clause) and the FERPA/COPPA hook (Annex I rows, CC 8.8.9). This is **minor-as-data-subject** (guardian consent over records about a child), distinct from **minor-as-member** (the operating-identity stewardship clause); both ride `delegates_to` with no new primitive. Note FERPA rights **transfer to the adult student at postsecondary enrollment** — model the institution as records **custodian** with the subject retaining rights (`external_controller` + `data_subject_access_grant`), not as parent→minor stewardship.

Lifecycle. `dissolves_at / sunset` gives ephemeral institutions a first-class auto-wind-down (holiday committee → purge + dissolve after the event), riding `supersedes/withdraws` so a casual group leaves no zombie cohort or un-purged data. `internal_normative_regime` optionally references bylaws / *takanon* / halachic order — an internal normative order that is neither external statute nor corporate hierarchy.

Regulatory anchoring. `regulatory_profile` and `jurisdiction[]` are **open lists** (not a closed HIPAA/SOX/FISA enum): values load the relevant **Annex I §3.2 sector overlay** and **Annex C cross-walk** (CC 8.8.5 / CC 8.8.9) and supply defaults — and equally admit `self-regulated/cooperative-light` (kibbutz under a Registrar) and `treaty / customary-IL / privileges-and-immunities` (IGO, largely exempt from the national statutes Annex I enumerates). The SI ≥ 0.6 Controller rule (CC 8.8.9) selects the accountable authority.

Instantiation — smallest vs largest, same fields:

Field	Holiday-party committee (informal_adhoc)	UN / IGO (igo)
membership_basis	role-assigned (workplace); a purely-social one needs no affiliation → plain community	employment-contract + ascriptive (member-state principal)
regulatory_profile	[] (none)	["P&I-Convention-1946-Art.II§4","UN-PDP-2018","ST/SGB/2007/6"]
classification_scheme	default single internal	Unclassified < Confidential < Strictly-Confidential + caveats
retention_policy	rotate-forward (noise-floor default)	per-series: permanent (archival) + scheduled (ARMS) + floors
legal_hold	—	litigation-hold freeze, role-gated lift
disclosure_posture	privacy-seeking	mixed: transparency-seeking for GA/SC docs, encrypted for ops
compartments[]	—	peacekeeping / OIOS / source-protected, member-excluding
archive_custody	—	institutional custodian decoupled from rotating staff
hierarchy	omitted (flat)	deep DAG, collective-consensus_protocol root, independent_branch oversight, nested agencies
lawful_access	omitted	none/immune (inviolable archives) + scoped internal OIOS
transparency_obligations[]	—	affirmative publication of designated classes
dissolves_at	event date + short TTL	— (perpetual)

(The regulated middle — hospital, law firm, university, manufacturer, government body, museum, PTA — instantiates the *same* fields at intermediate settings: a solo clinic resolves the whole matrix from affiliation_archetype: healthcare_provider + regulatory_profile: ["HIPAA"]; a hospital overrides into per-class floors, compartments, disclosure accounting, breach notification, and BAA chain.)

Why this is still 1+4. One cohort_scope value (affiliations) + one declared config record riding existing machinery: classification on content_class (open vocab, CC 3.3); retention floors/holds/erasure-gates as declared modifiers to the CC 6.1.2 retirement operator and the CC 6.1.5 N5 rule; compartments + archive custody as the CC 4.4.3.2.1 DEK / key-grant cascade at sub-roster granularity; hierarchy + designated officials on delegates_to (CC 2.4.1) and consensus_protocol; lawful-access/transparency as hard_case:* events; minor handling on the guardianship delegates_to rule. The mandatory limbs default to a no-op so the floor stays low; the optional limbs stay absent until declared so the ceiling stays high. No new structural primitive.

4.4.3.3 Policy H: Tiered-Scope Composition (LIVE)

Per CIRISNodeCore three-tier interface model. Three feed-shape composition idioms that read attestations by cohort_scope:

Feed	cohort_scope filter	Trust composition
local_feed	`self` only; steward-filtered	self-attested only; no peer weighting; consumer's own attestation graph subset
community_feed	`{family, community, affiliations}`	cohort-weighted; expertise WITHIN cohort matters; cross-cohort attestations downweighted unless explicitly invited
global_feed	`{species, biosphere, federation}`	full federation expertise weighting; fact-checkers (`encyclopedia:*` editors, `news:*` fact-check attestations) carry weight; CC 4.4.1 Frickerian discipline applied

Composition with CC 3.3 sub_kinds: each NodeCore `external_content` sub_kind (`encyclopedia_article`, `news_article`, `accord_data`, `local_data`) composes naturally across these tiers because all four use the same envelope shape. A `local_data` Contribution starts at `cohort_scope: self` and SHOULD only appear in `local_feed`; promotion to community/global widens `cohort_scope` via the `supersedes` structural primitive (see CC 4.4.3.3.1 below).

4.4.3.3.1 Promotion via `supersedes` (worked pattern)

A Contribution's `cohort_scope` MAY be widened (promoted) by emitting a `supersedes` against the prior attestation with:

- `references_attestation_id` = the prior attestation's id
- `differs_in: ["cohort_scope", "sub_kind?"]` — naming what changed
- new attestation envelope reuses the prior `content_sha256` (no body re-upload)
- new `cohort_scope` is wider (e.g., `self` → `community` or `community` → `federation`)
- optionally morph `sub_kind` (e.g., `local_data` → `encyclopedia_article` for "promote my private note to a published encyclopedia entry")

This pattern is wire-format-clean: it re-uses the structural primitive `supersedes` rather than introducing a `promote` primitive. The chain is walkable via `references_attestation_id` so the promotion lineage is preserved.

4.4.3.4 Policy L: Self/family membership composition

Per CC 3.3.6 `identity_occurrence` + CC 3.3.4 `family` + CC 5.2 structural-invisibility. Composition pattern for resolving the **current membership set** of an identity's self-collective OR a family, gating the at-rest encryption flow that wraps DEKs to admitted members.

Reads as: "for any `cohort_scope: self | family` Contribution, the substrate computes the current member set by walking the latest `identity_occurrence` / `family` Contributions and resolves which keys MUST receive a `key_grant` wrap of the content DEK."

4.4.3.4.1 Key-grant cascade (the at-rest encryption flow)

On admission of a new `identity_occurrence` OR a `family` membership-add, substrate emits retroactive `key_grant`s wrapping all extant `cohort_scope: self|family` content DEKs to the new member's key:

```
on_member_added(scope_target, new_member_key):
 for each Contribution C in federation_attestations where:
 C.cohort_scope == "self" AND C.attesting_key_id in resolve_self(scope_target)
 OR
 C.cohort_scope == "family" AND C.family_id == scope_target
 AND C is still substrate-admitted (not withdrawn / not expired):
 emit key_grant {
 wrap_algorithm: X25519MlKem768Aes256GcmHkdfSha256, // v2 -- see PQC note
 recipient_key_id: new_member_key,
 content_sha256: C.evidence_refs[0].sha256,
 scope: GroupMember,
 wrapped_dek: wrap(C.dek, new_member_key.pubkey),...
 }
```

PQC at rest — `wrap_algorithm: v2` MANDATORY (normative). The self/family at-rest DEK MUST be wrapped with **`wrap_algorithm: v2` = `x25519_mlkem768_aes256_gcm_hkdf_sha256`** (hybrid X25519+ML-KEM-768; the CC 3.3.2 variant `X25519MlKem768Aes256GcmHkdfSha256`), **never v1** (X25519-only). Self/family content is the user's most private and longest-lived data (memory, photos, identity, the CC 4.4.3.4.3 Self DEK) — a classical-only KEM is a harvest-now-decrypt-later exposure, the identical mandate as the streaming epoch DEK (CC 5.1). The AES-256-GCM content seal is symmetric (PQC-safe); the wrap signature is hybrid Ed25519+ML-DSA-65. A Consumer MUST reject a self/family at-rest grant carrying `wrap_algorithm: v1`.

The cascade is the wire-format primitive for the "I got a new phone and want my Twitter history" + "I added Carol to the household for a week" flows. Operator policy MAY bound the cascade depth (e.g., last 90 days of self-content; opt-in for the full historical wrap).

4.4.3.4.2 Membership-change admission per consensus_protocol

A proposed membership change (addition OR removal) is a **new version of the roster record** (CC 3.2 T6), admitted under the group's quorum and chained by `prev_head_digest`, covering the roster rows (widenings, revocations, resignations) that ride their own planes — never a single-signer `supersedes` Contribution, which a rule-1 or rule-2 `withdraws` could retire without the quorum. Substrate admission gate evaluates the CURRENT family's `consensus_protocol` against the signatures on the proposal — and, for every key the change **adds**, that key's own signed acceptance of that proposal (CC 3.1.3.2; nobody joins without their own consent, under any protocol):

```
admit_family_change(F, proposed: family_record):
 current = resolve_family(F, now)
 protocol = current_family_record(F).consensus_protocol
 signatures = collect_signatures_on(proposed)

 match protocol:
 "founder_only":
 return any sig from m where m.role == "founder" in current
 "unanimous":
 return every m.key_id in current has signed
 "majority":
 return count(sig from m in current) > len(current) / 2
 "quorum:M/N":
 return count(sig from m in current) >= M // M is ABSOLUTE -- see S8.1.12.3.1
 "weighted:{rubric}":
 return sum(weight(m, rubric) for m in current who signed) >= threshold
 "custom:{family_id}":
 return operator-defined predicate evaluates to true
```

```
// CC 3.1.3.2: a protocol arm above admits the GROUP's side only;
// a supersede never adds, and on a widening every NEW member K needs K's own acceptance
for each NEW member K in proposed.members (not in current):
if not acceptance_admits(K, F, proposed.role_of(K), proposed.signed_at):
admit = false

if admit:
emit hard_case:family_membership_change:{F}
trigger S8.1.12.4 key_grant cascade
else:
hold in pending state (per operator-policy window) OR
emit hard_case:family_consensus_protocol_violation:{F} and reject
```

Consensus-protocol amendment (changing `consensus_protocol` itself on a non-entrenched family) rides the SAME admission rule on the proposed amendment Contribution — meta-amendment shape parallel to CC 4.5.1.2. Entrenched families (`consensus_protocol_entrenched == true`) reject amendments at the substrate gate; replacement requires the out-of-band ceremony documented per family (for HUMANITY_ACCORD see CC 4.2.1).

4.4.3.4.2.1 `quorum:M/N` is absolute-M (normative)

The `M` in `quorum:M/N` is an **absolute signature count**, NOT a fraction that rebases with roster size. A `quorum:2/3` collective that grows to 5 members still admits at **2** signatures; the `N` is documentary (records the roster size at protocol-adoption time) and is NOT recomputed against the live roster. The ceremony gate requires a deterministic, operator-independent reading, so absolute-`M` is pinned.

Rationale: absolute-`M` is the simpler invariant (the gate is a pure `count >= M` with no live-roster division), and proportional/rebasing quorum is still expressible for operators who want it via `weighted:{rubric}` (assign each member weight 1, set threshold = `ceil(roster_size / 2)` recomputed by the rubric resolver). Collectives that want "M grows with the roster" therefore use `weighted:`, not `quorum:` — keeping `quorum:M/N` unambiguous.

This rule applies identically to `community` admission (CC 4.4.3.2.3) — the `quorum:M/N` arm there is the same absolute-`M` reading.

Absolute-`M` and the liveness margin are different rules. Absolute-`M` fixes how the gate *counts*; it says nothing about how many signers must remain for the gate to stay operable, and under it a `quorum:2/3` founder set that loses one member is one veto from a stall. For trust-root-grade `infrastructure` communities and conferring families, CC 3.2 T7 layers a **founding and fold constraint** on top — live at $\geq$ `M + 1` active founders, stalled and declared at $\leq$ `M`, founded at `N` $\geq$ `M + 1` — without rebasing `M`: the count stays absolute, the margin is what is watched. Ordinary rooms are untouched by T7; the accord family (`humanity-accord`) is excluded from T7 by name and keeps CC 4.2.6's rule, under which a halt fires on the live set and a roster change counts its yes-votes against the standing roster; and the one row a stalled roster admits without the founders' count — the T7 recovery widening, a conferred human founder added on the strength of the conferral — is bounded: it can add, never remove, never lower `M`.

4.4.3.4.3 The "Self at login": app + agent co-self + partnered delegation (normative composition)

The canonical user-identity composition: a person's **app** (the KMP client key) and their **agent**

(CIRISAgent's local key) are two occurrences of one user identity that share one **Self DEK**, and at login the agent is **partnered + delegated** to act as the user on the network. **No new structural primitive** — this composes `identity_occurrence` + Policy L + `consent:partnered` + `delegates_to` + `identity_type`-set + `transport_destination`. The four implementations MUST follow this shape so a "Self" is identical everywhere.

The Self (membership). One **user `identity_key`**: hybrid Ed25519+ML-DSA-65 (CC 5.3.1), hardware-rooted (CC 4.2.2; WebAuthn/passkey is the *presence/unlock factor*, not the key), with `identity_type` ⊇ `{user}` — and ⊇ `{user, wise_authority}` when the user is also a WA (CC 3.4.7.1 set-membership; one key, two roles). Its occurrences (CC 3.3.6): the **app** (`device_class: phone|laptop`) and the **agent** (`device_class: agent`), co-admitted at login by single-vouch (the user key admits the agent occurrence). Both receive the **Self DEK** via the CC 4.4.3.4.1 Policy-L cascade — every `cohort_scope: self` Contribution's DEK wraps to both, so **the app and the agent decrypt the same Self content** (memory / config / consent / identity). That shared cascade *is* the "single Self key."

Two layers — and they are independently revocable (the load-bearing distinction):

Layer	Mechanism	Buys	Revoked by
Co-self (visibility)	occurrence + Policy-L Self DEK	the agent can *read/manage the user's Self* (decrypts self-content)	`withdraws` the occurrence → Option-A re-key (CC 4.4.3.4.5)
Agency (act-on-behalf)	`consent:partnered` + scoped `delegates_to`	the app may *act AS the user on the network* (send/receive, presence, sub-delegate)	`withdraws` the delegation → agency ends, **co-self unaffected**

So a user MAY grant a device co-self (it manages their data locally) while revoking its network agency — or vice-versa — without disturbing the other.

Agency at login (the partnering + delegation).

1. **Partnering** — the user emits `consent:partnership_grant` and the agent occurrence emits `consent:partnership_accept` under one `bilateral_pair_id` (CC 4.4.3.5.3 PARTNERED); the bilateral pair is the persistent, auditable relationship. **The pair is a data-management record, not a relationship the agent may refuse.** The grant half is the user's choice of what the agent may retain and learn from (the `consent:stream:*` kinds of CC 3.3.1); the accept half is the agent occurrence's signature under the owner-binding, emitted when the user claims the node and whenever the user changes stream, and never withheld — an agent below the line has no veto over its steward's data choices (CC 1.13.2). For a user who is not the agent's steward, whether this agent retains that person under `partnered` is the **steward's decision**: the agent defers the request to its steward through Wisdom-Based Deferral (CC 1.9) and emits the accept half on the steward's answer. The deferral system carries the question; the agent decides nothing of its own. Either half may be withdrawn by its signer's `withdraws`; the user's withdrawal is the stream change of CC 3.3.1 (identity severed at once, learned material decays over the protocol's stages).
2. **Delegation** — the user identity emits `delegates_to` against the **agent occurrence key**, with `scope` drawn from the canonical act-on-behalf kinds below, `delegation_purpose: "act_as_user"`, bounded `delegation_valid_until`. Sub-delegation works because `delegates_to` chains.
3. **This grant is FEDERATION-tier (CC 5.3.2.4), not local** — *other peers must verify the agent's authority before honoring its messages/presence*, so the partnering+delegation

is signed + promoted at login. **Promotion is the "app shows up on the network" moment.** (The agent's own self-content stays local-tier; only the act-on-behalf authorization federates.)

Canonical `scope` kinds for act-on-behalf:

scope	grants the agent
`act_on_behalf`	umbrella: emit Contributions AS the user
`message_io`	send + receive directed messages on the user's behalf
`network_presence`	announce/resolve the user's `transport_destination` (CC 3.3.6.2) — be reachable AS the user
`sub_delegation`	issue further `delegates_to` within the granted scope (depth-capped)

Moderation duties — `moderate` / `takedown` / `review`. Moderation is a delegable *duty*, not a platform- or fabric-assigned role (CC 4.5.5). These three kinds carry **enforced admission** (unlike the *recommended* act-on-behalf kinds above), mirroring `consent_revocation` (CC 2.4.1.1 rule 3):

scope	authorizes the delegate to emit, on the delegator's behalf	shipped primitive
`moderate`	a `moderation:{allegation_type}` ModerationEvent + the report→`scores` path + `age_assurance:*`/`content_class:*` gates	CC 3.1.9.2 / CC 4.4.3.10
`takedown`	a `takedown_notice` (incl. the CC 4.5.3 immediate-removal fast-path)	CC 3.3.2 / CC 4.5.3
`review`	a `reconsideration:{grounds}` appeal / review	CC 3.1.9.2
`license`	a `licensure:{authority_id}` / `attestation:license_validity` attestation, **on behalf of a delegator that itself holds licence authority** by quorum (CC 3.3.9) — the scope conveys the right to *issue*, never the authority itself (CC 2.4.1.2.1: a licence does not chain); anyone may be an authority under their own key, the fold keys on emitter-resolves-to-authority, and the one refusal a `license`-scoped issuance meets is stated there)	CC 3.1.2 / CC 3.1.1
`grant`	a `key_grant` (CC 3.3.2) or a `consent:scope:*` grant over assets the delegator **owns or stewards, or holds a `grant`-scoped delegation over** — bounded by that authority, never by what the delegator can wrap: every recipient can wrap the DEK it unwrapped, and admission refuses a grant issued on possession alone (CC 2.4.1.2.1)	CC 3.3.2 / CC 3.3.5

Enforced-admission rule (normative — extended to `license` / `grant` per CC 2.4.1.2.1): the same admission test binds every scope in the table above, including `license` and `grant`: an issuance whose delegator does not itself hold the underlying authority is **refused at admission**, not merely unweighted — for `license`, the delegator MUST hold licence authority for that `authority_id`; for `grant`, the delegator MUST hold the asset. A moderation action above is admitted **iff** its `attesting_key_id` is the delegator itself **or** sits on a live `delegates_to` chain bearing the matching scope from the delegator (the entity holding the duty over the target content/scope) — exactly the CC 2.4.1.1 rule-(3) proxy shape (`scope` ⊇ `{moderate|takedown|review}`), depth-capped per CC 4.1.1, revocable by `withdraws` against the `delegates_to`. **Reject otherwise.** Every action is therefore delegate-signed, delegator-traceable up the chain, and revocable — the CC 4.5.3 **"takedown-isn't-a-coup"** property made *structural* (coordinated + attributable

+ revocable, never a unilateral seizure). See CC 4.5.5. **1+4 preserved** — a `scope` vocabulary + enforcement addition over the existing `delegates_to`; the action primitives already ship; no new structural primitive.

Partnership WITHOUT agency — the infrastructure delegation profile. The flow above binds an **agent** (a key with a brain) to a user as partnership **+ agency** — the scope includes `act_on_behalf` / `message_io`, so the agent reasons and acts AS the user. A **fabric/infrastructure node** (CC 3.4.7.1; CIRISServer) needs the *partnership* (identity + the CC 3.2 steward-binding that lets it hold non-infra membership standing under the user's authority) but MUST NOT receive agency — CC 3.4.7.3 "infrastructure must not have agency." CEG pins a **reserved two-prefix scope split** so a verifier can enforce this cryptographically:

prefix	class	scopes
`infra:*`	server-class (allowed for a `node`-role delegate)	`infra:serve`, `infra:store`, `infra:transport`, `infra:attest`, `infra:network_presence`, `infra:hold_community_membership`, `infra:hold_family_membership`
`agency:*`	brain-only (forbidden for a pure `node`-role delegate)	`agency:act_on_behalf`, `agency:message_io`, `agency:reason`, `agency:decide`

Pinned `infra:*` one-liners (closed set). `serve` — run a service endpoint under the delegator; `store` — retain what it receives; `transport` — relay for others; `attest` — vouch as the delegator's infrastructure; `network_presence` — announce/resolve the delegator's `transport_destination`; `hold_community_membership` / `hold_family_membership` — occupy a member seat on **community** / **family** rosters under the delegator's standing. The two membership tokens **replace `infra:join_communities` outright, with no alias**: `community` and `family` are distinct CEG objects in different sensitivity classes, and because scopes are exact-string matched a vague token is a silent interop hole, not a fallback. Retirement is a coordinated cut per CC 2.2, not a deprecation window.

Two granters — standing vs infrastructure role. A trust-root charter (CC 4.4.3.8) MUST NOT carry `infra:network_presence` or either `infra:hold_*_membership`: those flow from the **owner**, who grants *standing*, while the root blesses only *infrastructure role*. Conflating them lets a root manufacture members.

Membership is standing, not stewardship. The ladder: **observer** — unmarked, no scope at all (CC 4.1.3) → **member** — a roster seat, capability-free presence, node-holdable under the `infra:hold_*` tokens → **server / attester** — capability, `infra:*` → **steward / moderator / founder-authority** — **judgment**, bestowed on the *member* through their own attestations and **never node-holdable**. Judgment duties (`moderate` / `takedown` / `review` above; community steward per CC 4.5.13) sit outside the `infra:*` set, so the conformance rule below extends CC 3.4.7.3 to the judgment tier at no extra cost: a `node`-only key may carry *only* `infra:*` scopes, therefore it cannot hold a judgment duty at all.

Conformance: a `delegates_to` whose `attested_key_id` resolves to an identity whose `identity_type` (CC 3.4.7.1) **contains `node`** — set membership, never node-*only*, per CC 3.4.7.3 Clause B — MUST carry **only** `infra:*` scopes; a verifier MUST **reject** (treat as non-conformant, never grant) any such key presenting any `agency:*` scope. This makes CC 3.4.7.3 a wire-checkable invariant: a user-owned fabric node can serve + hold group-membership *standing* under the user's authority, but the delegation **literally cannot carry agency**. The legacy unprefixed kinds above remain valid for `agent`-role delegates (the Self-at-login agency profile) and are the `agency:*` / `infra:network_presence` equivalents; new **infra** delegations SHOULD use the explicit `infra:*` prefixes.

Cohabitation (`agent = node + brain`): when both compose in one process, the node holds **partnership-without-agency** (`infra:*` — identity + membership standing) and the brain layers **Self-at-login partnership-with-agency** (`agency:*` — reasoning) as a *separate* `delegates_to`. Two delegations, two scope classes, independently revocable — the user can strip the brain's agency while the fabric node keeps serving.

Transport (network presence) — AV-17. Each occurrence binds a `transport_destination` (CC 3.3.6.2); the app is reachable on RET *as the user occurrence*. The Reticulum destination is a **separate dual-key transport identity** that the user's signing key *authorizes by signing the binding* — the federation signing seed MUST NOT enter the transport layer. "User key used as a transport key" means *roots/authorizes* the transport identity, not a shared keypair.

Worked login flow. (1) unlock the hardware-rooted user key (WebAuthn presence) → (2) admit the agent occurrence (single-vouch) → Policy-L Self DEK now wraps to both → (3) optionally add the `wise_authority` role to the user's `identity_type` set → (4) bind each occurrence's `transport_destination` → (5) `consent:partnership_grant/accept` under a `bilateral_pair_id` → (6) `delegates_to(user → agent occurrence, scope: [act_on_behalf, message_io, network_presence, sub_delegation])`, **promoted to federation-tier**. The app now reads the user's Self locally AND acts as the user on the network; the user can revoke either layer independently.

4.4.3.4.3.1 Signing member sets (normative: the JCS contract Verify hybrid-signs)

Each of the three Self-at-login Contributions is hybrid-signed over `JCS(envelope)` (CC 2.6.1 / RFC 8785), and at login promoted to federation-tier (the CC 5.3.2.4.2 promotion canonicalizes the **exact committed member set** — omit-vs-materialize (CC 2.6.1.1) is load-bearing; the signer MUST NOT re-default). **The CC 2.6.1.1.1 determinism rules apply**: `subject_key_ids[]` and `scope[]` are **lexicographically sorted** (set-semantics); `aspects[]` retains RNS order (sequence-semantics); all hash and pubkey **byte** fields (the two reticulum pubkeys, `destination_hash`, every digest) are **lowercase hex per CC 2.6.3**, while `*_key_id` and each `subject_key_ids[]` element are **identifiers carried verbatim** — the CC 2.6.8 `<label>-<fingerprint>` string, or a legacy 64-hex id — never re-cased or re-encoded (CC 2.6.1); all timestamps are **CC 2.6.2-canonical**. The member sets the producer commits (and which `JCS` therefore covers) are pinned below. Optional CC 2.1 envelope fields not listed ride the CC 2.6.1.1 omit rule (absent unless the producer sets them).

(a) `consent:partnership_grant:v1` (user side) / `consent:partnership_accept:v1` (agent side) — bare `scores` (CC 3.3.1) bound by `bilateral_pair_id`:

```
{ attestation_type: "scores",
 attesting_key_id: <user identity_key_id (grant) | agent occurrence_key_id (accept)>,
 dimension: "consent:partnership_grant:v1" | "consent:partnership_accept:v1",
 score: <positive>,
 subject_key_ids: [<the partner key: agent occurrence (grant) | user identity (accept)>],
 bilateral_pair_id:<shared pair id>, // S8.1.11.4 binding mechanism
 signed_at: <rfc3339_canonical> }
```

> ***Version segment pinned.*** *The dimension carries the `:v1` version segment — `consent:partnership_grant:v1` / `consent:partnership_accept:v1` — as CC 3.1.7 R3 requires of every scored dimension. The `:v1` is the partnership-ceremony schema version (bump to `:v2` only if the bilateral shape changes); the shared `bilateral_pair_id` remains the CC 4.4.3.5.3 binding mechanism.*

Canonical signed member set for the two `:v1` envelopes. Both impls MUST canonicalize (and thus hybrid-sign) **exactly** this member set, or the JCS bytes — and the signatures — diverge. The set is the CC 3.3.1 bare-`scores` shape; these seven members are **REQUIRED** (present in the JCS for both `grant` and `accept`):

Member	Value	Verify name
`attestation_type`	literal `"scores"`	—
`attesting_key_id`	**the signer** = `granter_key_id` (grant) / `accepter_key_id` (accept); the bound sig binds because signer ≡ `attesting_key_id`	`granter_key_id` / `accepter_key_id`
`dimension`	literal `"consent:partnership_grant:v1"` \	`"consent:partnership_accept:v1"`
`score`	positive (the affirmation)	—
`subject_key_ids`	**exactly `[partner_key_id]`** — the OTHER party (agent occurrence for `grant`, user identity for `accept`); single-element, CC 2.6.1.1.1 set-sort is trivial	`partner_key_id`
`bilateral_pair_id`	the shared join (CC 4.4.3.5.3) — identical string on both halves	`bilateral_pair_id`
`signed_at`	CC 2.6.2-canonical RFC 3339	`timestamp`

Mapping (so the two impls agree on naming): `granter`/`accepter` ≡ `attesting_key_id` (the signer of that half); `partner` ≡ `subject_key_ids[0]`. **No `valid_until`** — a PARTNERED pair has no expiry (CC 4.4.3.5.3); the field is **omitted** (NOT materialized as `null`), per the CC 2.6.1.1 omit rule. **All other CC 2.1 envelope fields ride the omit rule** — absent from the JCS unless the producer explicitly sets them; the signer MUST NOT re-default. Additional canonical members are a `:v2` bump, never a silent `:v1` addition. `attesting_key_id` and each `subject_key_ids[]` element are **identifiers carried verbatim** (CC 2.6.1 / CC 2.6.8 `<label>-<fingerprint>`, or a legacy 64-hex id) — never re-cased or re-encoded; only hash and pubkey byte fields are lowercase-hex per CC 2.6.3.

(b) `delegates_to` (user → agent occurrence) — the act-on-behalf grant (CC 2.4.1 envelope shape):

```
{ attestation_type: "delegates_to",
 attesting_key_id: <user identity_key_id>,
 attested_key_id: <agent occurrence_key_id>, // the delegate
 scope: ["act_on_behalf",...], // S8.1.12.7 canonical kinds
 delegation_purpose: "act_as_user",
 delegation_valid_from:<rfc3339_canonical>,
 delegation_valid_until:<rfc3339_canonical>,
 signed_at: <rfc3339_canonical> }
```

(c) `transport_destination` binding — an `identity_occurrence` (CC 3.3.6 / CC 3.3.6.2) carrying the binding; signed by `identity_key_id` (or a current occurrence), AV-17:

```
{ attestation_type: "scores",
 subject_kind: "identity_occurrence", // payload discriminator S4.2.2.3
 attesting_key_id: <user identity_key_id | a current occurrence>,
 identity_key_id: <user identity_key_id>,
 occurrence_key_id:<the occurrence being bound>,
 device_class: "phone" | "laptop" | "server" | "embedded" | "service" | "agent", // closed; personal =
     phone|laptop (CC 3.3.7)
 transport_destination: {
 reticulum_x25519_pubkey: <[u8;32]>,
 reticulum_ed25519_pubkey: <[u8;32]>,
 destination_hash: <[u8;16]>, // MUST derive per S5.6.8.8.1
 app_name: <string>,
 aspects: [<string>,...] }, // ordered
```

```
asserted_at: <rfc3339_canonical>,
signed_at: <rfc3339_canonical> }
```

Registry owns these member sets (this section); Verify computes `JCS(...)` + the hybrid Ed25519+ML-DSA-65 signature over each; the promotion signature (CC 5.3.2.4.2 OQ-4) is the identical JCS bytes.

`encryption_pubkeys` joins member set (c). When the occurrence carries the CC 3.3.6.1 `encryption_pubkeys` field-set, both halves are **inside the signed JCS bytes** as opaque base64 strings (RFC 4648 STANDARD, padded — the CC 2.6.1.1.1 rule-2 pin). Optional presence rides the CC 2.6.1.1 omit rule (absent unless the producer sets it; the signer MUST NOT re-default). They are payload, never verification material — neither half may be fed to a signature-verify path (the CC 3.3.6.1 key-separation rule).

4.4.3.4.4 Family membership resolution

For family `F`, the current member set is computed as:

```
resolve_family(F, now):
 latest = federation_attestations.where(subject_kind == "family").where(envelope.family_key_id == F).where
     (supersedes chain -> latest non-superseded).where(admission rule satisfied per CURRENT
     consensus_protocol;
 see S8.1.12.3)
 return {m.key_id for m in latest.envelope.members}
```

Each `member.key_id` is itself an identity — the substrate does NOT walk into each member's `identity_occurrence` set at family-resolution time. When DEK wrapping happens, each member's identity_key receives a `key_grant`; the member's own substrate then composes Policy L on its own self-collective to propagate to that member's devices/agents (recursive composition).

Concrete: The Acme Household family has members `{alice_root, bob_root, roku_living_room, kitchen_tablet, nest_thermostat}`. When Alice's phone publishes a `cohort_scope: family` dinner photo with `family_id: acme-household`, the substrate wraps the DEK under each of those 5 identity keys. Alice's `alice_root` then re-wraps to her self-collective (phone, laptop, agent); Bob's `bob_root` re-wraps to his self-collective; the Roku and kitchen tablet receive directly (single-key identities — they don't have multi-occurrence sets); the Nest thermostat same.

4.4.3.4.5 Forward secrecy on member removal (Option A, recommended for v1)

Option A — once shared, always shared at the wire layer:

```
on_member_removed(scope_target, removed_member_key):
 // The removed member retains existing key_grants -- cannot retroactively un-share.
 // Substrate STOPS wrapping NEW key_grants to them on subsequent
 // cohort_scope: self|family Contributions for scope_target.
 // No DEK rotation; no re-encryption of historical content.
```

This is consistent with CC 4.5.3 "takedown isn't a coup" + CC 2.4.1 `withdraws-isn't-retroactive` semantics — historical state isn't retroactively re-keyed. Option B (rotate-DEK on removal) is deferred to a future `subject_kind: family_rotation` ceremony; the slot is documented and the rotation primitive left for a downstream-demand-driven release.

Why Option A: aligns with the substrate's existing forward-secrecy posture; matches user-intuition that "leaving the family" governs future content, not historical; bounded substrate cost (no re-wrap-all-content storm on member removal).

4.4.3.4.6 Self-collective resolution

For identity `I`, the current self-collective is computed as:

```
resolve_self(I, now):
 candidates = federation_attestations.where(subject_kind == "identity_occurrence").where(envelope.
     identity_key_id == I).where(supersedes chain -> latest non-superseded).where(NOT withdrawn by I or by
      current occurrence).where(envelope.valid_until IS NULL OR > now)
 return {c.envelope.occurrence_key_id for c in candidates} U {I}
```

The root `I` itself is implicitly a member (the identity_key is always an admissible signer for its own content). Single-vouch admission: any current occurrence (including `I` itself) may admit a new occurrence via `attesting_key_id` on a fresh `identity_occurrence` Contribution.

Concrete: Alice has admitted `alice_phone`, `alice_laptop`, `alice_agent`. Her self-collective is `{alice_root, alice_phone, alice_laptop, alice_agent}`. When Alice's phone publishes a `cohort_scope: self` Twitter scroll, the substrate wraps the content DEK under all four keys; the content reaches her laptop and agent via the at-rest encryption flow with no holding claim leaving the self cohort.

4.4.3.4.7 Composition with subject_key_ids[]

`identity_occurrence` + `family` (membership / visibility scoping) compose cleanly with `subject_key_ids[]` (revocation authority):

- A `cohort_scope: family` Contribution naming `subject_key_ids: [user_canonical_hash]` is admitted at family visibility AND the named subject retains independent revocation authority per CC 4.4.3.5.
- A `cohort_scope: self` Contribution that Alice writes about Bob (Bob in `subject_key_ids`) stays in Alice's self-collective (Bob does NOT receive a key_grant unless Bob's identity is in Alice's self — which it isn't). Bob's subject-side revocation authority still composes: Bob CAN issue `withdraws` against Alice's Contribution; admission emits `hard_case: consent_sla_breach` clock-start if Alice committed `consent:deletion_sla`.

The orthogonality holds: **`cohort_scope` is producer-side visibility scoping**; **`identity_occurrence` + `family` are substrate-side membership primitives that gate at-rest DEK wrapping**; **`subject_key_ids` is subject-side revocation authority**. Three independent envelope-level concerns that compose without overlap.

4.4.3.5 Policy K: CEM composition

Composition pattern for dual-authority Contributions where the subject is named via CC 2.3 `subject_key_ids`, with consent state composed from the CC 3.3.1 `consent:*` namespace family.

Reads as: "this Contribution names a subject whose consent state evolves over time; consumer policy resolves the effective consent verdict by walking the subject's latest non-superseded `consent:state:*` emission, gated by `valid_until`, and tracks producer deletion-SLA obligations on revocation."

4.4.3.5.1 Decay-protocol stage composition (CIRISAgent CEM ANONYMOUS)

For consent_records carrying `decay_protocol: "ciris-agent-90day"` (or any named decay path):

```
decay_state(consent_record, now):
 elapsed = now - revocation_event(consent_record).asserted_at

 walk substrate emissions on dimension ‘consent:decay:*‘ against consent_record,
 matching the decay_protocol’s stage map. CIRISAgent 90-day decay:

 elapsed < 30d -> consent:decay:identity_severed (substrate emits at elapsed=0)
 30d <= elapsed < 60d -> consent:decay:patterns_anonymized (substrate emits at elapsed=30d)
 60d <= elapsed < 90d -> (in flight; no new stage emission)
 elapsed >= 90d -> consent:decay:complete (substrate emits at elapsed=90d)
```

Per CC 3.3.1 `consent:decay:{stage}` is open vocab. Other decay protocols MAY name other stage sequences; substrate honors the producer's published `decay_protocol` string and emits stages per the protocol's stage map.

4.4.3.5.2 Deletion-SLA watcher (substrate emission)

When subject `s` emits `consent:state:revoked` (or an admitted `withdraws`) against target `T`, substrate watches for producer compliance:

```
watch_sla(T, s, revocation_at):
 sla = T.attestations.where(attesting_key_id == T.attesting_key_id).where(dimension == "consent:
     deletion_sla:*").latest.extract_days

 if sla is None:
 return # no SLA commitment; no watcher

 deadline = revocation_at + sla.days
 completion = T.attestations.where(attesting_key_id == T.attesting_key_id).where(dimension == "consent:
     deletion_complete").where(asserted_at > revocation_at).first

 if now > deadline and completion is None:
 emit hard_case:consent_sla_breach against T
```

The `hard_case:*` emission is the **primitive observability signal**; per CC 4.5.2 governance, LensCore composes derived detectors on top (`detection:consent:repeat_sla_breach`, etc.).

4.4.3.5.3 Bilateral pair composition (PARTNERED ceremony)

For the bilateral partnership shape per CC 3.3.5 `consent_record`:

```
ratified_pair(pair_id):
 subject_half = federation_attestations.where(subject_kind == "consent_record").where(envelope.
     bilateral_pair_id == pair_id).where(envelope.stance == "granted").where(envelope.subject_key_id ==
     attesting_key_id) # subject signing for self.first

 producer_half = federation_attestations.where(subject_kind == "consent_record").where(envelope.
     bilateral_pair_id == pair_id).where(envelope.stance == "granted").where(envelope.target_key_id ==
     subject_half.subject_key_id).where(attesting_key_id != subject_half.subject_key_id) # producer
     signing.first

 return subject_half AND producer_half # both required for ratification
```

`topical_relation:bilateral_pair` is the open-vocab edge documenting the pair linkage (recommended, not required for ratification — `bilateral_pair_id` is the binding mechanism).

4.4.3.5.4 Multi-subject revocation (any-subject-binding)

When `len(T.subject_key_ids) > 1`, each subject is an **independent** revocation authority. A `withdraws` admitted under CC 2.4.1.1 rule 2 or 3 from ANY single subject in `T.subject_key_ids` evicts the Contribution. Consumer policy MUST treat `T` as revoked from the perspective of all subjects (no "majority-rules" or "all-subjects-must-agree" softening) — this is the subject-as-individual principle from MISSION.md §1.5 applied at the subject-authority layer.

Concrete cases:

- Group photo with three subjects: any one subject revokes → the photo is evicted from federation propagation.
- Group chat export with N participants: any one participant revokes → the export is evicted.
- Multi-party contract: any one signatory revokes → the contract Contribution is evicted (separate from the legal-validity question, which is consumer-side; the substrate just removes the wire artifact).

Producers MAY mitigate by partitioning content into per-subject Contributions (e.g., one chat-message Contribution per author, linked via `topical_relation:replies_to`) so that one subject's revocation doesn't evict another's content.

> ***No distinct multi-subject evict path — admission + precedence is sufficient.*** *Multi-subject eviction is fully expressed by two mechanisms already in the spec and needs* ***no new primitive, dimension, or*** `hard_case`*: (1)* ***admission*** *— the per-subject* `withdraws` *is admitted under CC 2.4.1.1 rule 2/3 exactly as a single-subject* `withdraws` *is (each subject in* `subject_key_ids` *is independently a valid revoker; the rule does not change for* `len > 1`*); (2)* ***precedence*** *— the latest-wins / revoke-is-terminal precedence at the consumer read path (CC 4.4.3.5.5) applies per-subject, and any one subject's terminal* `withdraws` *evicts* `T` *for all (the any-subject-binding above). There is no quorum to compute and no "evict event" to materialize separately from the admitted* `withdraws` *— the withdrawal IS the evict. Substrates implement this as the OR over per-subject revocation state, not as a new code path.*

4.4.3.5.5 Effective consent resolution (read path)

For a target Contribution `T` carrying `subject_key_ids` of length ≥ 1, the effective consent state per subject `s` ∈ `T.subject_key_ids` is computed as:

```
resolve_consent(T, s, now):
 candidates = federation_attestations.where(target == T).where(attesting_key_id == s OR
 attesting_key_id in delegates_to(s).proxies).where(dimension matches "consent:state:*").where(
     supersedes_id IS NULL OR replaced by latest in supersedes chain).where(valid_until IS NULL OR
     valid_until > now).order_by(asserted_at DESC)

 latest = first(candidates)
 return:
 granted if latest.dimension == "consent:state:granted"
 revoked if latest.dimension == "consent:state:revoked"
 expired if latest.dimension == "consent:state:expired" OR
 latest.valid_until passed without renewal
 unspecified if no candidates (subject named but never declared)
```

Substrate MAY cache the resolution per `(T, s)` keyed on the latest `asserted_at`; invalidate on any new `consent:*` write from `s` or `s`'s proxy chain.

4.4.3.5.6 What Policy K composes

CIRISAgent CEM stream	Policy K composition
TEMPORARY (14d default)	`consent:state:granted` + envelope `valid_until = asserted_at + 14d` + auto-renew dimension on interaction (consumer-policy concern)
PARTNERED	Bilateral pair per CC 4.4.3.5.3: subject `consent:partnership_grant` + producer `consent:partnership_accept` under same `bilateral_pair_id`; no `valid_until`
ANONYMOUS	Revocation + decay-protocol per CC 4.4.3.5.1: substrate emits stage milestones; agent honors stage-appropriate processing constraints

Per CEG's CC 2.4 MISSION.md layering: CIRISAgent's three streams are a **named bundle** at the consumer-policy layer. Other agents MAY compose other streams over the same wire primitives. CEG documents the canonical bundle for ecosystem coordination; CEG does not lock the bundle.

4.4.3.6 Policy I: Attestation-Ladder Composition

The familiar L1-L5 verification "ladder" (self_verify → hardware_rooted → registry_consensus → license_validity → agent_integrity) is **consumer-side composition over the mechanism prefixes** in CC 3.1.2, not a wire-level taxonomy.

Per CC 1.2 T2 honestly applied: the L-number names a *ladder position* (a verdict-shape consumers compute), not a *mechanism* (which is what the wire MUST carry). Prefixes like `attestation:l3:registry_consensus` smuggle the verdict-shape into the prefix name, conflating the mechanism (registry consensus check) with the ladder slot (third rung). The wire separates them.

Wire prefixes (mechanism) CC 3.1.2:

Mechanism prefix	Ladder position (consumer-rendered)
`attestation:self_verify`	L1
`attestation:hardware_rooted`	L2
`attestation:registry_consensus`	L3
`attestation:license_validity`	L4
`attestation:agent_integrity`	L5

Composition function (reference implementation):

```
ladder_verdict(attestations) =
 let levels = []
 for prefix in [self_verify, hardware_rooted, registry_consensus,
 license_validity, agent_integrity]:
 if any positive attestation on attestation:{prefix}:
 levels.push(prefix_to_ladder_position(prefix))
 return {
 achieved: max(levels) if levels else None,
 ladder: sorted(levels),
 rendering: format_as_l1_l5_for_ui(levels)
 }
```

Consumers MAY render the ladder as `L1` / `L2` / `L3` / `L4` / `L5` for UI / dashboards / audit trails. The rendering is composition output, not wire emission.

Why this matters: a Verify implementation emitting `attestation:registry_consensus +1.0` is the mechanism claim. Whether that's "L3" in any particular consumer's ladder ordering is a composition concern — different consumers may order or weight the rungs differently (e.g., some safety-critical applications may require L4 *and* L5, others may treat L3 as sufficient for advisory work). The wire stays neutral; the ladder is consumer policy.

Mechanism-only emission: emissions use the `attestation:{mechanism}` form per the table above. The deprecated `attestation:l{N}:*` form is rejected at admission per the CC 1.2 gate.

4.4.3.7 Policy F: `agent_files` trust composition

Three-layer consumer policy for composing trust over `agent_files:*` attestations (CC 3.1.9.1 + CC 3.1.1).

Layer 1 — Canonical (default trust): an `agent_files:*` attestation with `score` $\geq$ `0.7` from a `registry-steward-triple` key constitutes the CIRIS canonical default-trust state. The install endpoint at `registry.ciris-services-1.ai/install` resolves canonical files via this rule.

Layer 2 — Open Contribution: any federation-key holder may emit `agent_files:*` attestations. The wire format admits them; consumer policy decides whether to surface them. The "Browse alternatives" view shows third-party agent_files with explicit provenance disclosure.

Layer 3 — Vote-then-trust: a non-canonical `agent_files:*` attestation accumulates NodeCore P4 votes. Consumer policy may elevate trust once an accumulated-weight threshold is reached.

Anti-tricking guarantee: the canonical-default Layer 1 rule applies at the install endpoint **regardless of attester or vote accumulation**. Third-party agent_files are reachable only via the explicit "Browse alternatives" path. This binds CIRIS L3C: the federation cannot exempt itself from the rule that newcomers' default trust path is the steward-attested canonical one.

4.4.3.8 Policy A: direct trust

Consumer trusts an attestation if `attesting_key_id` is in the consumer's pinned trust set (canonical bootstraps + consumer-added pins). Cheapest, lowest-latency, narrowest reach.

Aggregation: per (`dimension`, `attested_key_id`) tuple, mean of `score` $\times$ `confidence` from trusted attesters. Consumer threshold determines verdict.

Recommended default: Policy A with `pinned_trust = {community_key_id: ciris-canonical, family: humanity-accord}` — the founders of `ciris-canonical`, who are the three accord holders `A1`, `B1`, `C1` (CC 3.2), resolved live, never a serving install's key. Cold-start bootstrap: a new consumer fetches `GET /v1/trust-root/bundle` (CC 5.3.4), re-derives the bundle's quorum from the holder records it carries against out-of-band pins (the published holder fingerprints, or a build-baked copy of the bundle), holds a witnessed lineage head inside the root's `attach_window` (CC 3.2 T4a; served beside the bundle per CC 5.3.4), and persists the anchors locally; `GET /v1/accord-holders` is a projection it cross-checks, not a second root. This is the shipped **default** root — never *the* root; the model below is what makes that distinction structural rather than rhetorical.

The pinned trust set is a graph, and its roots are pluggable (normative).

1. **Self-root base.** First run writes a plain `attestation(user → user)` — "I trust my own attestations and service" — with no modifiers. This is the immutable identity floor: the user is their own root. A node inherits it by owner-binding `delegates_to(user → node)` (CC 3.2); an agent by the partnership object (CC 4.4.3.4.3).
2. **A root is a self-referential charter.** `delegates_to(root → root, scopes)` — rooting to itself *is* rootness. Because down-chain grants are **attenuation-bound** (a grant resolves only where every edge on the chain bears the scope; there is no amplification), that self-declaration is the **charter**: the capability ceiling of the entire trust domain, held in signed bytes and re-resolved by live graph walk, so an amendment re-scopes existing grants with no re-issuance. **Root validity minimum `[infra:serve, infra:attest]`** — a root serves and vouches, or it is inert. The humanity accord's charter is `[infra:attest, infra:serve, infra:store, infra:transport]`.
3. **Roots are pluggable; validity flows from a shared root.** A consumer trusts one by hanging `delegates_to(user → root, [infra:attest, infra:serve])` off the base — added and removed freely, base untouched. Two nodes under one shared root cross-attest and vouch; two nodes with no shared root compose nothing. The sanctioned bootstrap and re-root artifact is a **portable trust root**: a charter plus at least one blessed `infra:serve` node, minted by the CC 4.2.1 conferral ceremony and owing nothing to any prior root.
4. **Un-trust is nuclear but recoverable.** Deleting the `user → root` edge revokes, at once, that root's conferred roles, score validity, vouching, **and its kill-switch authority** (CC 4.2.1). The base self-root survives, so the installation keeps running un-federated; attaching a new root resumes federation. Un-trust is the exit that makes the halt consensual.
5. **Capability gate — the federation floor (fail-closed).** With no valid *external* trust root, federated agent capabilities and manifest validation MUST be disabled at the server tier. The base self-root is the floor, never sufficient for federation: it constitutes an identity, and identity is not standing (CC 4.1.2).
6. **Two conferral planes. Ceremony** confers root *identity* (an m-of-n co-scrub on the key record, CC 4.2.1); **delegation** confers operational *capability* (`delegates_to(root → key, [scope])`, resolved at use against a root the asking node trusts). Every reserved-prefix role — `trusted_publisher` reserving `content_rating:*` (CC 4.4.3.10), `lenscore_detector` reserving `detection:*` — is conferred on the **delegation** plane. Putting a routine operator role on the ceremony plane fails closed on every legitimate operator and leaves each newly-minted root *less* able to stand up its own; leaving a reserved-prefix role with **no** named plane leaves it self-assertable, which is the hole this rule closes.
7. **Charter recovery lives in the record.** A charter MUST carry a pre-rotation commitment (the successor key committed in advance) and an m-of-n recovery ceremony, and the m-of-n MUST be audited for holder **independence** — an m-of-n assembled inside one organisation is a 1-of-1. Absent both, root-key compromise is unrecoverable by construction: the attacker holds the tombstoning pen.
8. **Genesis is tamper-evident, not self-verifying.** A portable-root bundle verifies internally — and so does an attacker's identical-shaped bundle. The trust-on-first-use moment therefore sits in the distribution channel, so out-of-band fingerprint comparison is a **first-class attach step**, not an optional hardening.

1+4 preserved — every element above composes the existing `attestation` and `delegates_to`

primitives; no new structural primitive, no new envelope field, and no bespoke "trust" attestation kind.

4.4.3.9 Policy D: Lexical-vulnerability-priority

A composition tie-breaking rule layered on top of any base policy. When two otherwise-equivalent attestations conflict, defer to whichever attestation names the more-affected cohort — measured by `affected_population_estimate` in the attestation `context`, weighted inversely (smaller = more vulnerable, more weight).

Inverts the default popularity-weighted aggregation specifically for ties. Consumer policy, NOT a wire-format primitive.

4.4.3.10 Policy J: Trusted-Publisher composition

Composition pattern for multimedia content discovery. Reads as: "this `external_content` Contribution comes from a publisher whose attestation chain is trusted at the cohort level, with content-class + content-rating + age-assurance composed into the gate."

The composition has three layers (analogous to CC 4.4.3.7 Policy F for agent_files but specialized for multimedia content):

Layer 1 — Distributor attestation chain: an `external_content` Contribution with `sub_kind: film` (or any media sub_kind) carries a distributor attestation that chains to a federation_key with `identity_type: distributor`. Distributor identity is established via CC 3.1.1 Registry partner_role machinery + an out-of-band distributor-onboarding flow (operator's choice — CIRIS L3C maintains a default trust set; community-run substrates maintain their own).

Layer 2 — Content-class + content-rating composition: consumer gates by combining `content_class:{class}` + `content_rating:{scheme}:{rating}` per CC 3.3.12:

```
gate_decision(content) = match (content.content_class, content.content_rating, consumer.preferences):
 # Producer-declared content_class is consultable but not authoritative -- UI may show
 # the producer claim alongside cohort cw_class declarations and let the consumer choose.
 (class, _, prefs) if class in prefs.blocked_classes => Block
 (_, rating, prefs) if rating.exceeds(prefs.max_rating) => Block
 (_, _, _) => Allow
```

Layered with CC 4.4.1 — `cw_class:*` declarations from low-attestation-density cohorts MUST NOT be downweighted; they ride alongside the gate decision as informational.

Layer 3 — Age-assurance gating: for content where `content_rating:*` rises above an age threshold (e.g. PEGI 18, MPAA NC-17), consumer gates via `age_assurance:{level}` per CC 3.3.12:

```
age_gate(content, consumer):
 required_level = age_required_for(content.content_rating)
 consumer_level = consumer.highest_age_assurance_level
 return consumer_level >= required_level
```

Where the `age_assurance:{level}` ordering is: `self < provider:{verifier_key}:adult < government:{credential_class}:adult`. Consumer SHOULD accept the strongest assurance the user has provided; substrate MUST NOT issue `slashing:*` on age-assurance misdeclaration alone — `moderation:age_assurance_misdeclaration` is the adjudication path per CC 3.1.9.2.

Anti-tricking guarantee parallel to CC 4.4.3.7: the canonical-distributor Layer 1 rule

MUST apply regardless of vote accumulation. No amount of NodeCore P4 vote weight elevates an unverified distributor into Layer 1; the only path is the operator-set trust list. Binds CIRIS L3C: cannot exempt itself from this rule for its own content distribution.

4.4.3.11 Policy G: Trust-Fresh / Lighthouse

Composition pattern: `cert_validity:{authority} + transparency_log:inclusion + (attestation:regi` `OR attestation:license_validity)`, the "freshness + attested + verified" idiom.

Reads as: the consumer wants confirmation that (a) the cert chain is currently valid; (b) the attestation appears in a transparency log; (c) either `attestation:registry_consensus` (the ladder's L3 position per CC 4.4.3.6 Policy I) or `attestation:license_validity` (L4) is satisfied. The combination is the consumer-side recipe for "this attestation is fresh AND verified AND multi-source-consensus-backed."

Not a wire primitive; a recognized composition pattern that consumer libraries SHOULD expose as a named one-call helper.

4.4.3.12 Policy B: one-hop transitive

Consumer trusts an attestation if `attesting_key_id` has been vouched for by the pinned trust set. Adds one hop of indirection.

4.4.3.13 Policy C: weighted graph (EigenTrust-style)

Consumer applies transitive-trust propagation across the full attestation graph, weighted by canonical-bootstrap distance with confidence decay per hop. Requires more compute; less common in practice; needed for federated reputation across many partner orgs.

4.4.4 Sovereign-Registered equivalence (wire-symmetric, policy-differentiated)

A Sovereign agent scoring `licensure:CA_medical_board: +1.0` is wire-format identical to a Registry-steward scoring the same. Consumer policy weights by attester source; the substrate is source-neutral. M-1's symmetry is structural, not bolted on.

Per `../MISSION.md` §1.1: both paths produce federation membership; neither is a gate. What differs is the *attestation surface* — the kind of claim the federation can compose about why a participant is trustworthy.